" a b c b c a b b a c b a "

0 1 2 3 4 5 6 7 8 9 10 11

Palin = [b , b , a , a , c , c , b , b] --> first ouput

0 1 2 3 4 5 6 7

Palin = [b , c , a , b , b , a , c , b] --> second output

7 5 3 1 0 2 4 6 (sorted)

First Draft (logic):

```
palin_even = []
palin_odd = []
final_palin = []
final_palin.append(palin_odd)
final_palin.append(palin_even)
for k in range(len(palin)):
    if k%2 == 0:
        palin_even.append([k])
    elif k%2 != 0:
        palin_odd.append([k])

while median_left>0 and median_right<len(x)-1:
    if x[median_left] == x[median_right]:
        palin.append(x[median_left])
        palin.append(x[median_right])
    else:
        pass
```

" a b b b a "

0 1 2 3 4

palin = [b , b , b , a , a]

2 1 3 0 4

palin = [b , b , b , a , a]

0 1 2 3 4

final_palin = [a , b , b , b , a]

0 1 2 3 4

Second Draft:

```
x = "abcbcabbacba"
#x = "abbba"
def median(x):
  if len(x) % 2 == 0:
    return len(x)//2
  elif len(x) % 2 != 0:
    return (len(x) - 1)//2
  else:
    return None

palin = []

if len(x)%2 == 0:
    print("even length")
    median_left = int(median(x))
    median_right = int(median(x))+1

    while median_left > 0 and median_right < len(x) - 1:
      #print(median_left,median_right)
      #print(x[median_left],x[median_right])
      #median_right += 1
      #median_left -= 1
      if x[median_left] == x[median_right]:
        #print(x[median_left],x[median_right])
        palin.append(x[median_left])
        palin.append(x[median_right])
        median_left -= 1
```

```
        median_right += 1
        #print(x[median_left],x[median_right])
      else:
        median_left -= 1
        median_right += 1
    print(palin)
    #palin_even = []
    #palin_odd = []
    final_palin = []

    for k in range(len(palin)-1,0,-2):
          print(k,palin[k])
          final_palin.append(palin[k])
    for l in range(0,len(palin),2):
          print(l,palin[l])
          final_palin.append(palin[l])

    #final_palin.append(palin_odd)
    #final_palin.append(palin_even)
    result = "".join(final_palin)
    print(result)
    #print(final_palin)
#elif len(x) % 2 != 0:
#       print("odd length")
#       median_left = int(median(x))-1
#       median_right = int(median(x))+1
#
#       while median_left >= 0 and median_right <= len(x) - 1:
#           print(median_left,median_right)
#           print(x[median_left],x[median_right])
#           median_right += 1
#           median_left -= 1
#           if x[median_left] == x[median_right] and x[median_left]
== x[median_right] == x[median(x)]:
#               #print(x[median_left],x[median_right])
#               palin.append(x[median_left])
#               palin.append(x[median_right])
#               palin.append(x[median(x)])
#               median_left -= 1
#               median_right += 1
#               #print(x[median_left],x[median_right])
#           elif x[median_left] == x[median_right] and
x[median_left] == x[median_right] != x[median(x)]:
#               #print(x[median_left],x[median_right])
#               palin.append(x[median_left])
#               palin.append(x[median_right])
```

```
#                palin.append(x[median(x)])
#                median_left -= 1
#                median_right += 1
#            else:
#                median_left -= 1
#                median_right += 1
#
#        print(palin)

#print(median(x))
#print(median_left,median_right)
```

Third Draft:

```
n = int(input())
x = input()

def median(x):
  if n % 2 == 0:
    return n//2
  elif n % 2 != 0:
    return (n - 1)//2
  else:
    return None

palin = []
final_palin = []

if n % 2 == 0:
    median_left = int(median(x))
    median_right = int(median(x))+1

    while median_left > 0 and median_right < n - 1:
      if x[median_left] == x[median_right]:
        palin.append(x[median_left])
        palin.append(x[median_right])
        median_left -= 1
        median_right += 1
      else:
        median_left -= 1
        median_right += 1
    for k in range(len(palin)-1, 0, -2):
        final_palin.append(palin[k])
    for l in range(0, len(palin), 2):
        final_palin.append(palin[l])

    result = "".join(final_palin)
    print(len(final_palin))
```

```
    print(result)

elif n % 2 != 0:
    median_left = int(median(x))-1
    median_right = int(median(x))+1
    palin.append(x[median(x)])
    while median_left >= 0 and median_right <= n - 1:
      if x[median_left] == x[median_right]:
        palin.append(x[median_left])
        palin.append(x[median_right])
        median_left -= 1
        median_right += 1
      else:
        median_left -= 1
        median_right += 1

    middle = palin[0]
    pairs = palin[1:]

    left = []
    right = []

    for i in range(0, len(pairs), 2):
        left.append(pairs[i])
        right.append(pairs[i+1])

    left.reverse()

    final_palin = left + [middle] + right
    result = "".join(final_palin)
    print(len(final_palin))
    print(result)
```

Input:

```
x = "abcbcabbacba"
x = "abbba"
x = "malayalam"
```

Output:

```
abbba
PS D:\C> python -u "c:\Users
8
bcabbacb
PS D:\C> python -u "c:\Users
9
malayalam
PS D:\C>
```

Test Cases(Public):

*All public test cases passed, but failed in many private test cases due to -->It has only fixed centre to check for palindromes, centre should be variable checking anywhere where palindrome forms.

Save as Draft | Compile & Run | Submit | Reset

Compilation : Passed

Public Test Cases: 2 / 2 Passed

Note: These tests may not be considered while scoring. Know more.

Public Test Cases	Input	Expected Output	Actual Output	Status
Test Case 1	5 abbba	5\n abbba	5\n abbba\n	Passed after ignoring Presentation Error
Test Case 2	12 abcbcabbacba	8\n bcabbacb	8\n bcabbacb\n	Passed after ignoring Presentation Error

Fourth Draft(final):

*little tweaking by Claude (Sonnet 4.6) by Anthropic

```
n = int(input())
x = input()

palin = []
final_palin = []
result = x[0]  # assume first character is the longest palindrome initially

for center in range(n):

    # ====================
    # ODD LENGTH PALINDROME
    # ====================
    palin = []
    final_palin = []
    palin.append(x[center])        # middle character
    median_left = center - 1       # one step left of center
    median_right = center + 1      # one step right of center

    while median_left >= 0 and median_right <= n - 1:
        if x[median_left] == x[median_right]:
            palin.append(x[median_left])
            palin.append(x[median_right])
            median_left -= 1
            median_right += 1
        else:
            break                  # stop expanding if mismatch
```

```
    # rebuild palindrome from palin list
    middle = palin[0]
    pairs = palin[1:]
    left = []
    right = []
    for i in range(0, len(pairs), 2):
        left.append(pairs[i])
        right.append(pairs[i+1])
    left.reverse()
    final_palin = left + [middle] + right
    candidate = "".join(final_palin)

    # update result if longer or lexicographically smaller
    if len(candidate) > len(result) or \
       (len(candidate) == len(result) and candidate < result):
        result = candidate

    # ======================
    # EVEN LENGTH PALINDROME
    # ======================
    palin = []
    final_palin = []
    median_left = center            # left center
    median_right = center + 1       # right center

    if median_right <= n - 1:       # make sure right is within bounds
        while median_left >= 0 and median_right <= n - 1:
            if x[median_left] == x[median_right]:
                palin.append(x[median_left])
                palin.append(x[median_right])
                median_left -= 1
                median_right += 1
            else:
                break               # stop expanding if mismatch

        # rebuild palindrome from palin list
        for k in range(len(palin)-1, 0, -2):
            final_palin.append(palin[k])
        for l in range(0, len(palin), 2):
            final_palin.append(palin[l])
        candidate = "".join(final_palin)

        # update result if longer or lexicographically smaller
        if len(candidate) > len(result) or \
           (len(candidate) == len(result) and candidate < result):
            result = candidate

print(len(result))
print(result)
```

Test Cases(Public):
All Passed!

Test Cases(Private):
All Passed!

Save as Draft | Compile & Run | Submit | Reset

Private Test cases used for Evaluation	Status
Test Case 1	Passed
Test Case 2	Passed
Test Case 3	Passed
Test Case 4	Passed
Test Case 5	Passed
Test Case 6	Passed
Test Case 7	Passed
Test Case 8	Passed
Test Case 9	Passed
Test Case 10	Passed

;)